

Artificial Intelligence (AI-2002)

ADVERSARIAL SEARCH AND GAMES

Mr. Saif ur Rehman (Lecturer – AI & DS)

saifurrehman@nu.edu.pk

A decorative light blue triangle is located in the bottom right corner of the slide.

- What is the difference between a single-agent and a multi-agent environment?
- How do cooperative and competitive environments differ?

ADVERSARIAL SEARCH

- In which we explore environments where other agents are plotting against us.
- competitive environments, in which two or more agents have conflicting goals, giving rise to adversarial search problems.

Game Theory

- Game theory is the study of how agents make strategic decision.
- Adversarial agents with the techniques of adversarial game-tree search.
- We begin with a restricted class of games and define the optimal move and an algorithm for finding it: **minimax search**.
- Pruning makes the search more efficient by ignoring portions of the search tree that make no difference to the optimal move.

Game Theory

- For nontrivial games, we will usually not have enough time to be sure of finding the optimal move (even with pruning); we will have to cut off the search at some point.
- For each state where we choose to stop searching, we ask who is winning. To answer this question we have a choice: we can apply a heuristic evaluation function to estimate who is winning based on features of the state.

Game Theory

Stanford

Game Theory



Instructors: [Matthew O. Jackson](#) +2 more

Enroll for Free
Starts Mar 4

Financial aid available

551,590 already enrolled

Two-player zero-sum games

- The games most commonly studied within AI (such as chess and Go) are what game theorists call deterministic, two-player, turn-taking, perfect information, zero-sum games.
- “Perfect information” is a synonym for “fully observable,” and “zero-sum” means that what is good for one player is just as bad for the other: there is no “win-win” outcome.
- For games we often use the term move as a synonym for “action” and position as a synonym for “state.”

Two-player zero-sum games

- We will call our two players MAX and MIN.
- Position MAX moves first, and then the players take turns moving until the game is over.
- At the end of the game, points are awarded to the winning player and penalties are given to the loser

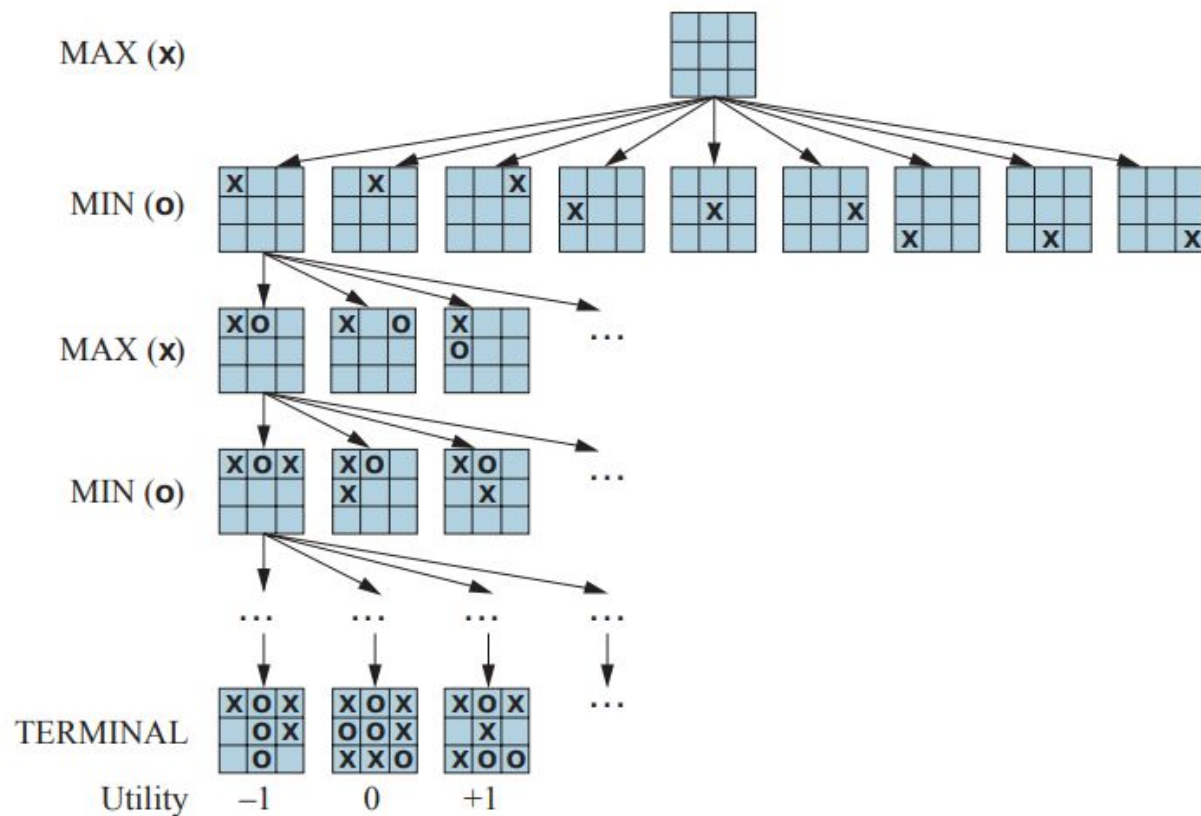


Figure 6.1 A (partial) game tree for the game of tic-tac-toe. The top node is the initial state, and MAX moves first, placing an X in an empty square. We show part of the tree, giving alternating moves by MIN (O) and MAX (X), until we eventually reach terminal states, which can be assigned utilities according to the rules of the game.

Two-player zero-sum games

- Figure shows part of the game tree for tic-tac-toe.
- From the initial state, MAX has nine possible moves.
- Play alternates between MAX's placing an X and MIN's placing an O until we reach leaf nodes corresponding to terminal states such that one player has three squares in a row or all the squares are filled.
- The number on each leaf node indicates the utility value of the terminal state from the point of view of MAX; high values are good for MAX and bad for MIN (which is how the players get their names).

Two-player zero-sum games

game can be formally defined with the following elements:

- S_0 : The **initial state**, which specifies how the game is set up at the start.
- $\text{TO-MOVE}(s)$: The player whose turn it is to move in state s .
- $\text{ACTIONS}(s)$: The set of legal moves in state s .
- $\text{RESULT}(s, a)$: The **transition model**, which defines the state resulting from taking action a in state s .
- $\text{IS-TERMINAL}(s)$: A **terminal test**, which is true when the game is over and false otherwise. States where the game has ended are called **terminal states**.
- $\text{UTILITY}(s, p)$: A **utility function** (also called an objective function or payoff function), which defines the final numeric value to player p when the game ends in terminal state s . In chess, the outcome is a win, loss, or draw, with values 1, 0, or $1/2$.² Some games have a wider range of possible outcomes—for example, the payoffs in backgammon range from 0 to 192.

Two-player zero-sum games

- We define the complete **game tree** as a search tree that follows every sequence of moves all the way to a terminal state.
- The game tree may be infinite if the state space itself is unbounded or if the rules of the game allow for infinitely repeating positions.

Minimax Search

- The Minimax Search is a decision-making algorithm used in adversarial search problems, especially in two-player turn-based games like chess, tic-tac-toe, and checkers.
- It helps a player (MAX) to choose the best possible move while assuming the opponent (MIN) plays optimally to minimize MAX's advantage.

Minimax Search

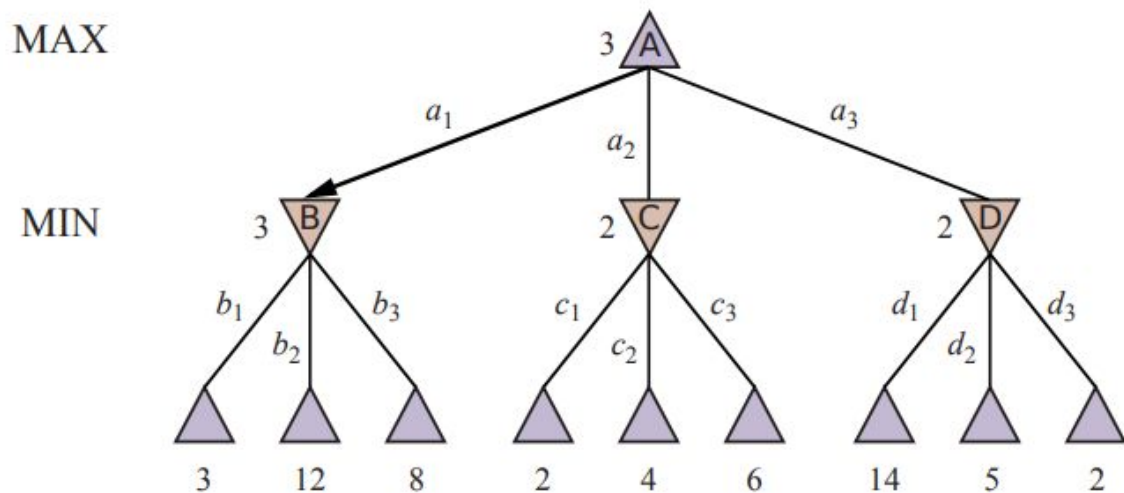


Figure 6.2 A two-ply game tree. The $\triangle$ nodes are “MAX nodes,” in which it is MAX’s turn to move, and the ∇ nodes are “MIN nodes.” The terminal nodes show the utility values for MAX; the other nodes are labeled with their minimax values. MAX’s best move at the root is a_1 , because it leads to the state with the highest minimax value, and MIN’s best reply is b_1 , because it leads to the state with the lowest minimax value.

Minimax Search

- Consider the trivial game in Figure 6.2. The possible moves for MAX at the root node are labeled a1, a2, and a3.
- The possible replies to a1 for MIN are b1, b2, b3, and so on.
- In some games, the Ply word “move” means that both players have taken an action; therefore the word ply is used to unambiguously mean one move by one player, bringing us one level deeper in the game tree.
- The utilities of the terminal states in this game range from 2 to 14.

Minimax Search

- Given a game tree, the optimal strategy can be determined by working out the minimax value of each state in the tree.
- which we write as $\text{MINIMAX}(s)$.
- The minimax value is the utility (for MAX) of being in that state, assuming that both players play optimally from there to the end of the game.
- The minimax value of a terminal state is just its utility.

Minimax Search

- In a nonterminal state, MAX prefers to move to a state of maximum value when it is MAX's turn to move, and MIN prefers a state of minimum value (that is, minimum value for MAX and thus maximum value for MIN). So we have:

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MIN} \end{cases}$$

Minimax Search

- The minimax algorithm performs a complete depth-first exploration of the game tree.
- If the maximum depth of the tree is m and there are b legal moves at each point, then the time complexity of the minimax algorithm is $O(b^m)$.
- The space complexity is $O(bm)$.

Minimax Search

- The minimax algorithm performs a complete depth-first exploration of the game tree.
- If the maximum depth of the tree is m and there are b legal moves at each point, then the time complexity of the minimax algorithm is $O(b^m)$.
- The space complexity is $O(bm)$.

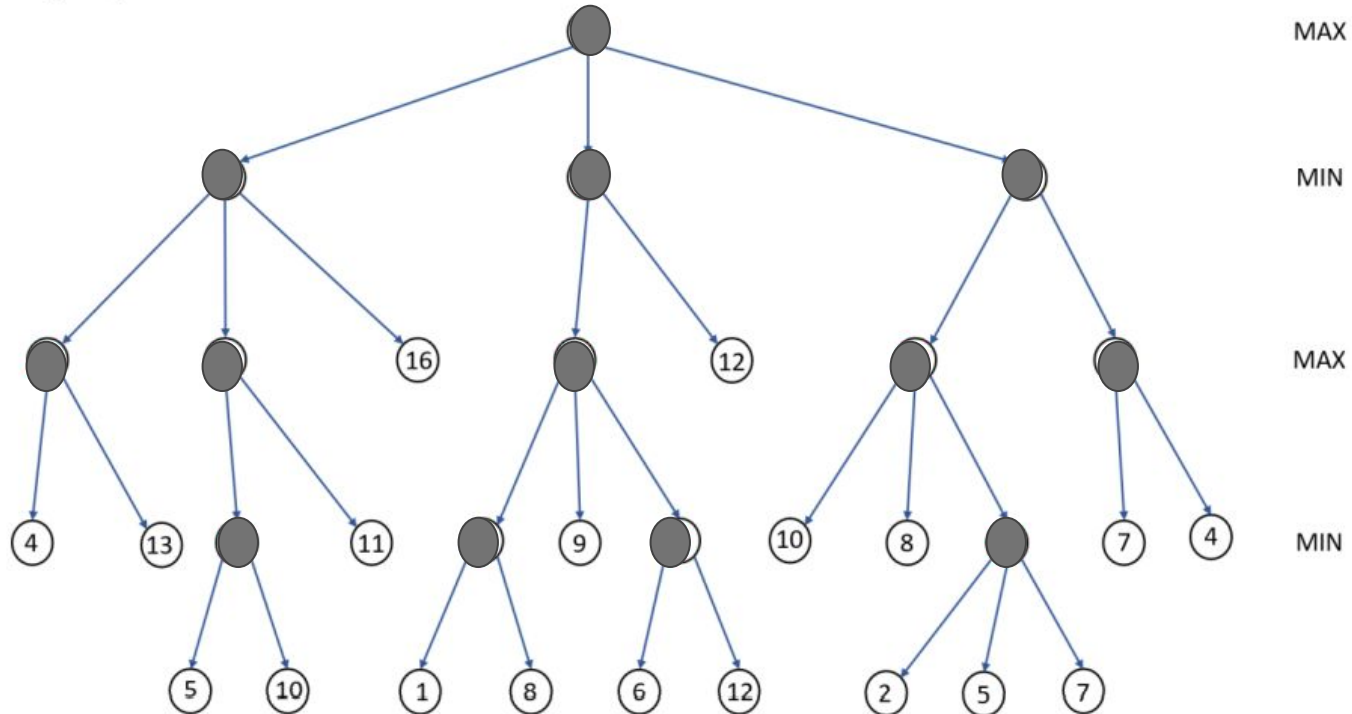
Minimax Search

- The exponential complexity makes MINIMAX impractical for complex games.
- Chess has a branching factor of about 35 and the average game has depth of about 80 ply, and it is not feasible to search $35^{80} \approx 10^{123}$ states.

Problem 1: [15] Minimax and Alpha-Beta

(a) [4] Use the Minimax algorithm to compute the minimax value at each node for the game tree below.

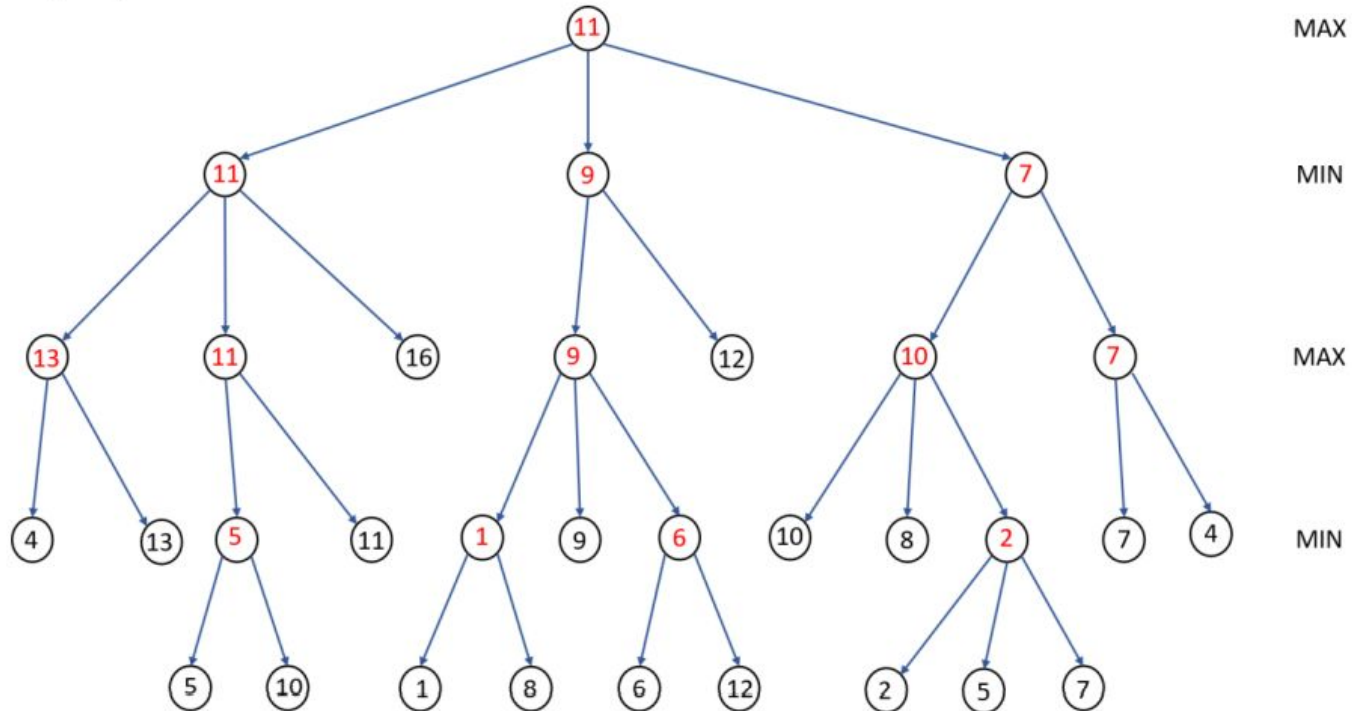
[Ans]:



Problem 1: [15] Minimax and Alpha-Beta

(a) [4] Use the Minimax algorithm to compute the minimax value at each node for the game tree below.

[Ans]:



Alpha–Beta Pruning

- The number of game states is exponential in the depth of the tree.
- We can sometimes cut it in half, computing the correct minimax decision without examining every state by pruning large parts of the tree that make no difference to the outcome.
- The particular technique we examine is called alpha–beta pruning.

Alpha-Beta Pruning

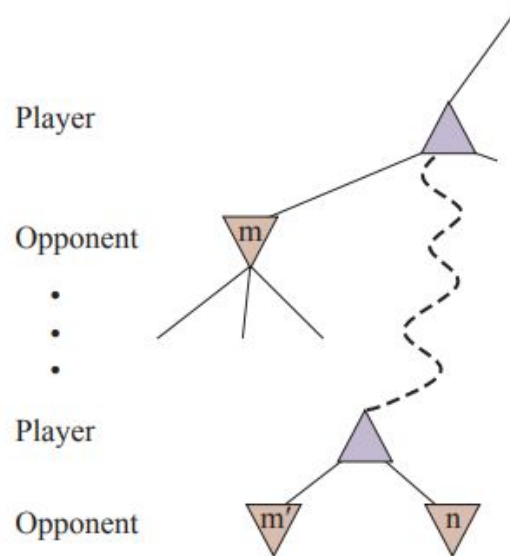
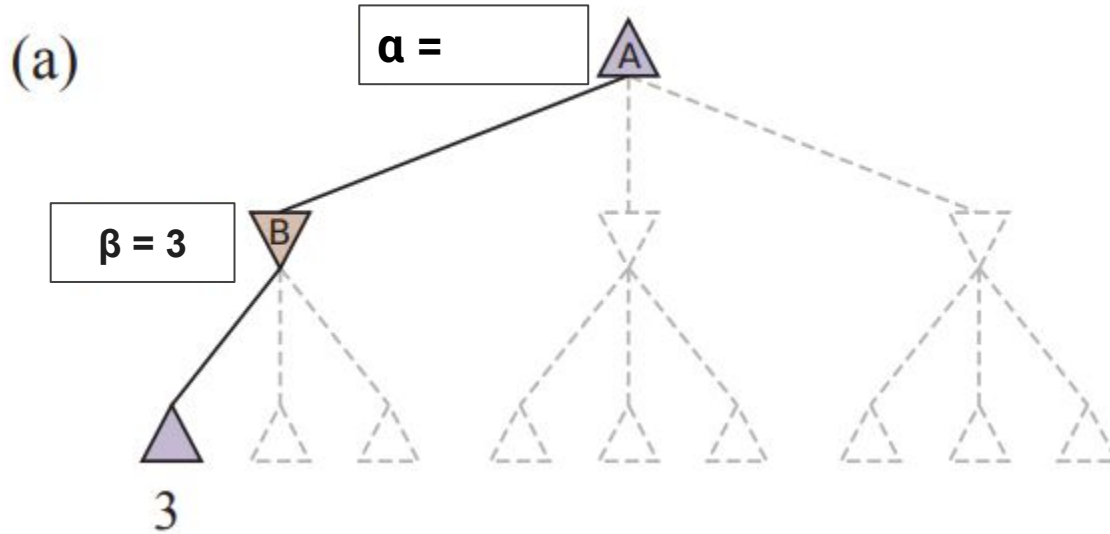
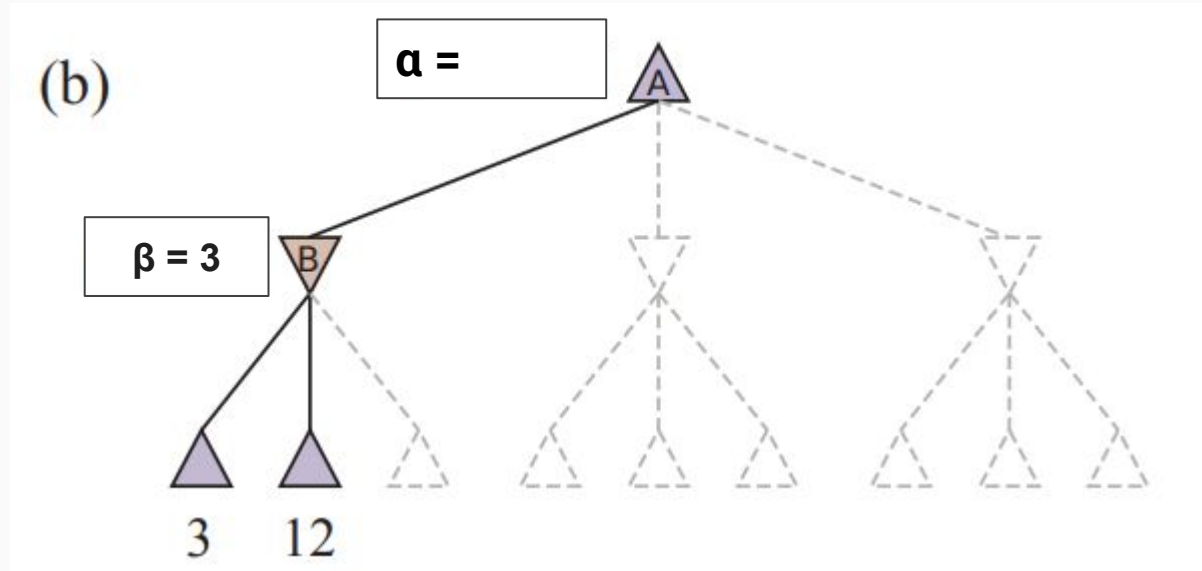


Figure 6.6 The general case for alpha-beta pruning. If m or m' is better than n for Player, we will never get to n in play.

Alpha-Beta Pruning

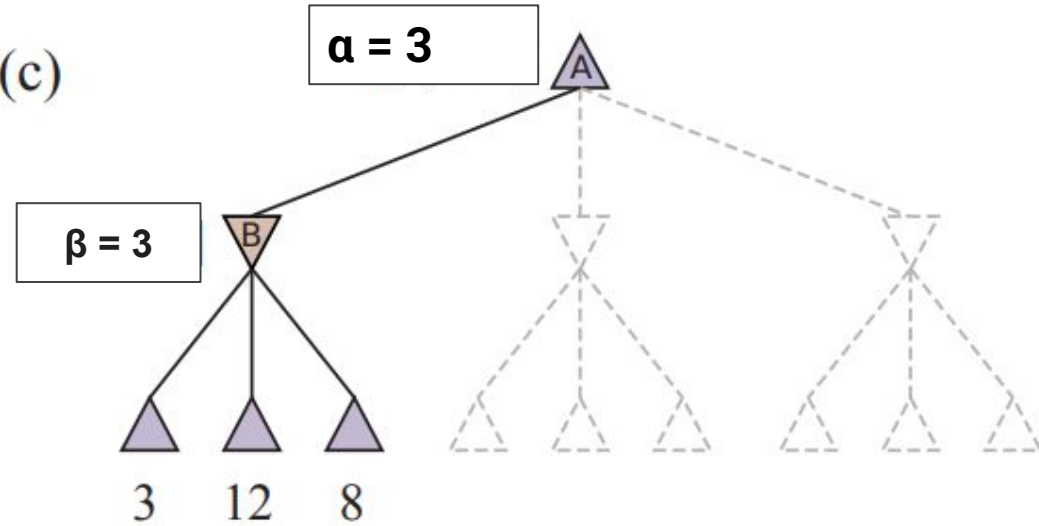


Alpha-Beta Pruning

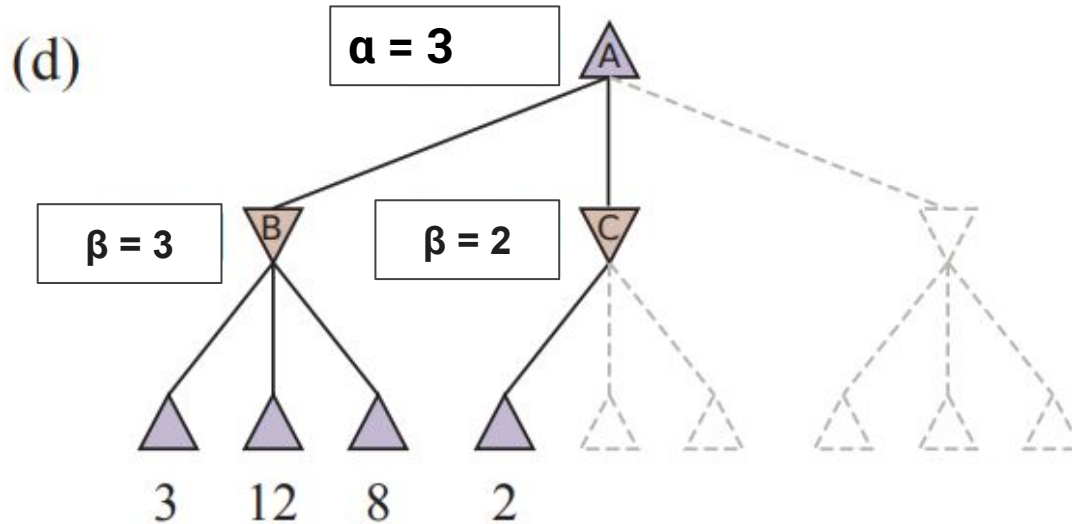


Alpha-Beta Pruning

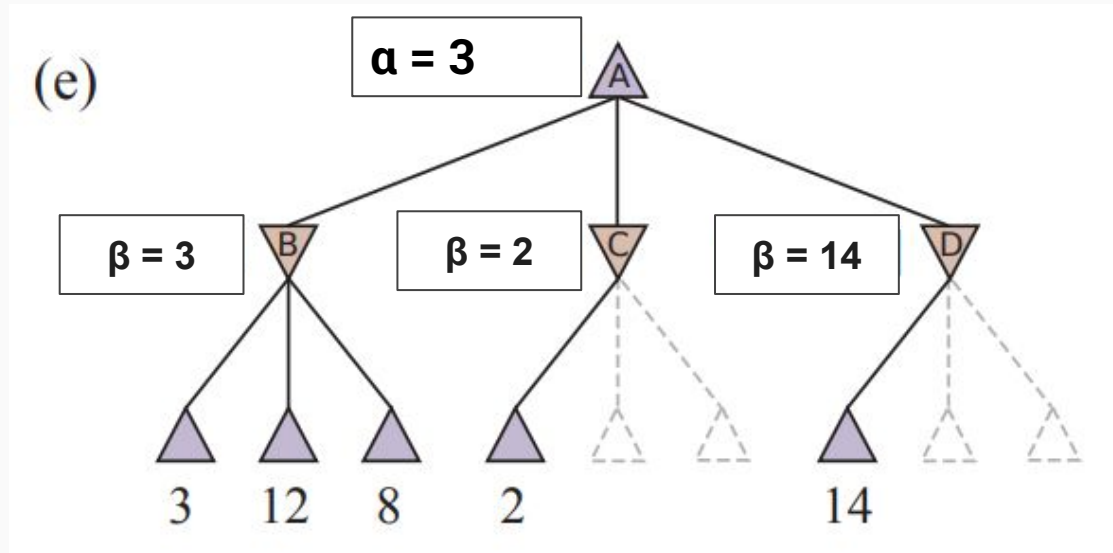
(c)



Alpha-Beta Pruning

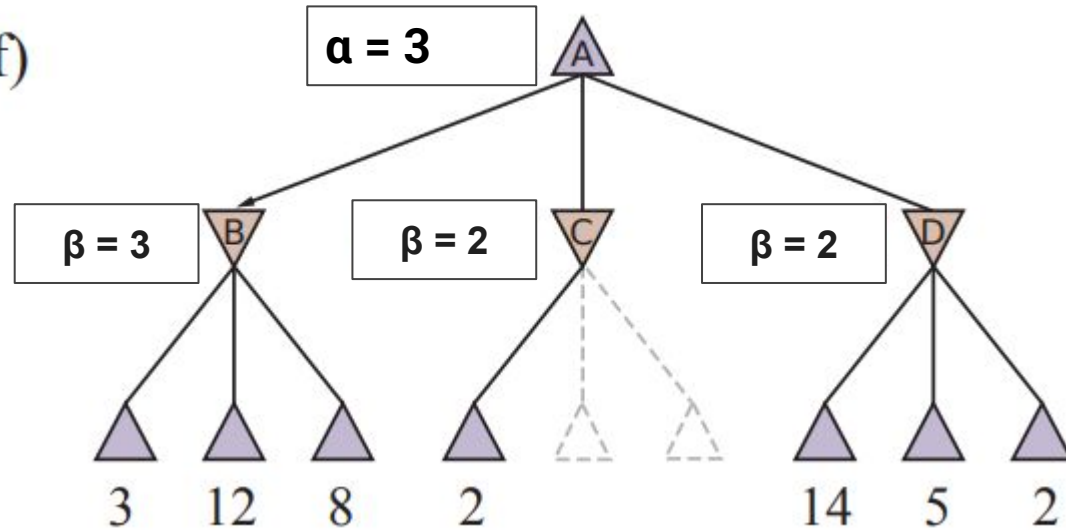


Alpha-Beta Pruning

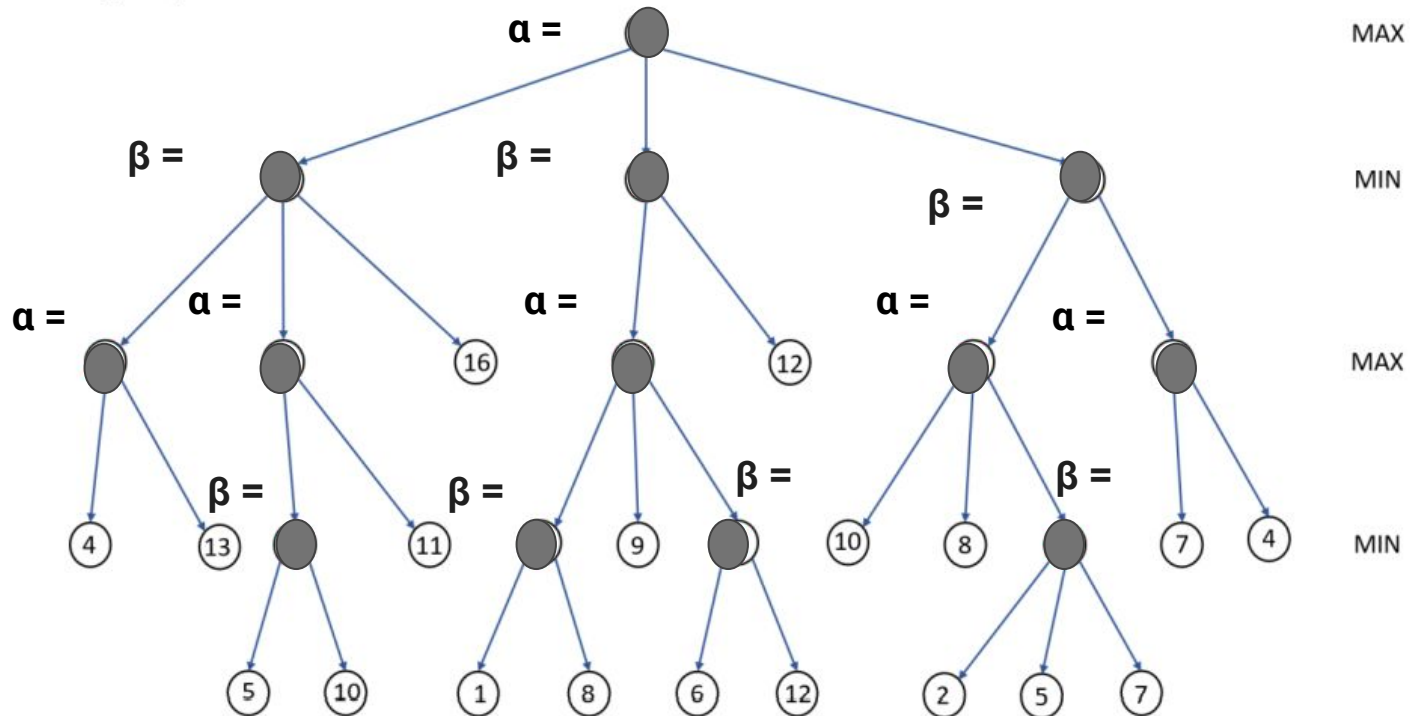


Alpha-Beta Pruning

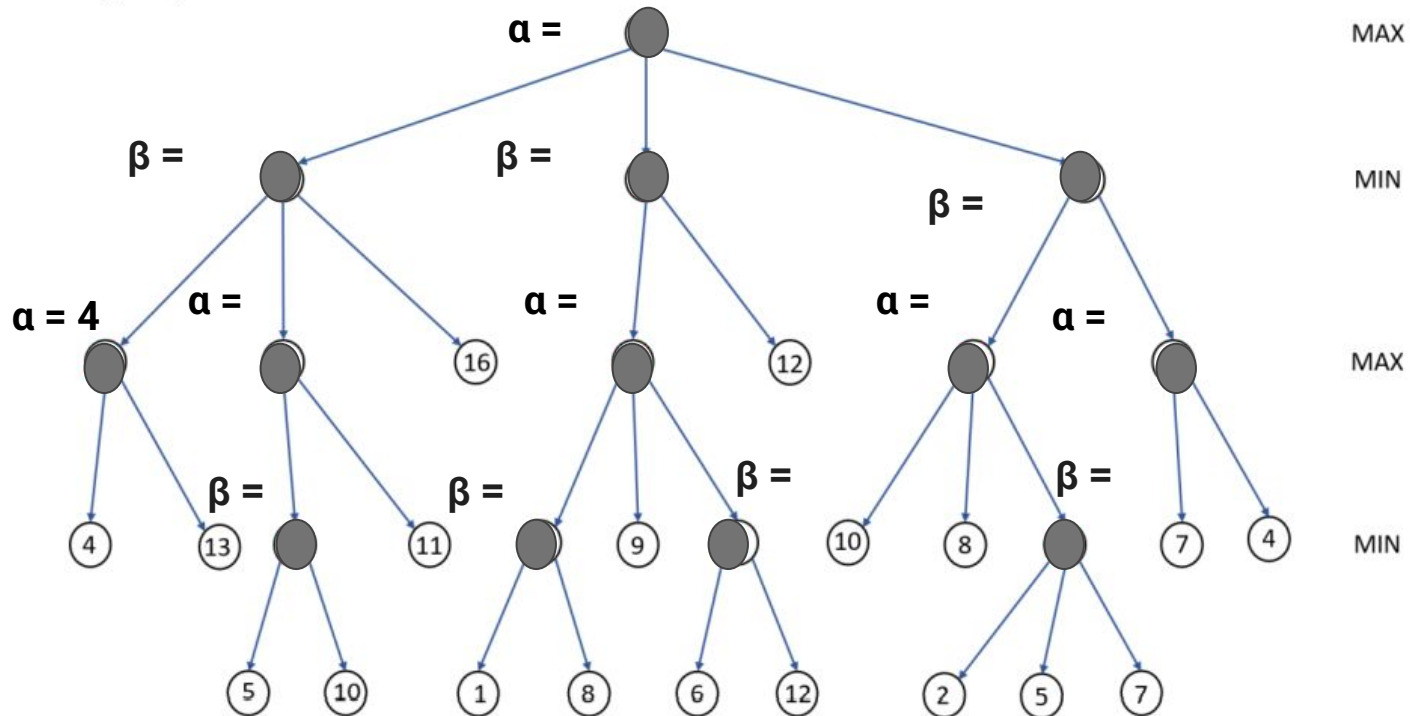
(f)



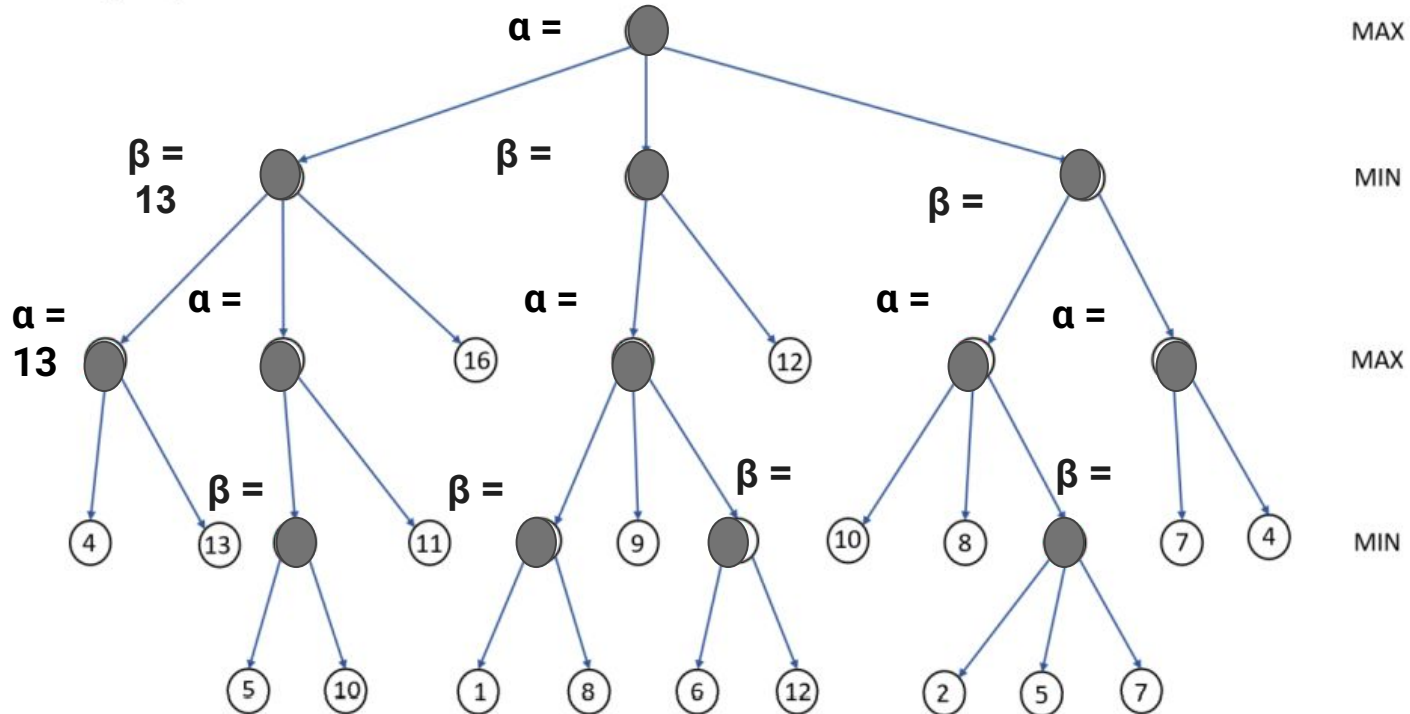
Use the Alpha-Beta pruning algorithm to prune the game tree below. Assuming child nodes are visited from left to right.



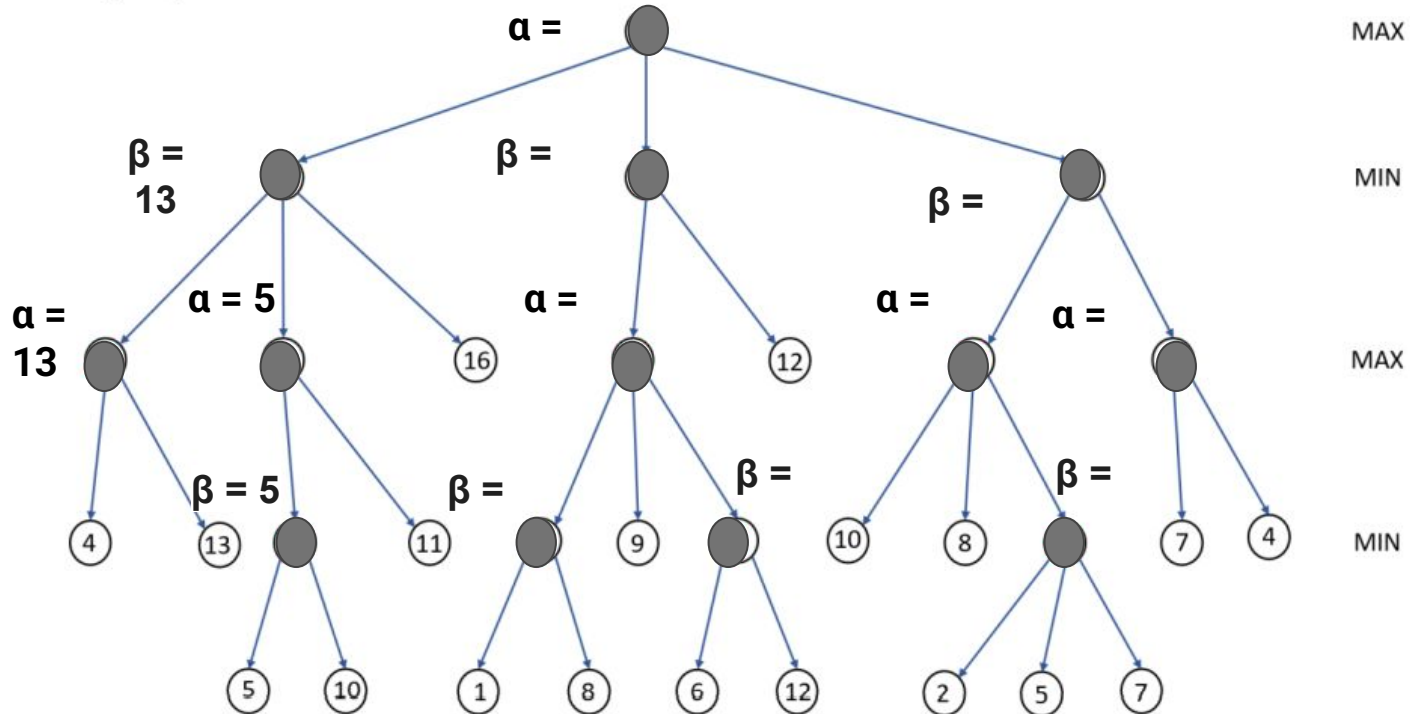
Use the Alpha-Beta pruning algorithm to prune the game tree below. Assuming child nodes are visited from left to right.



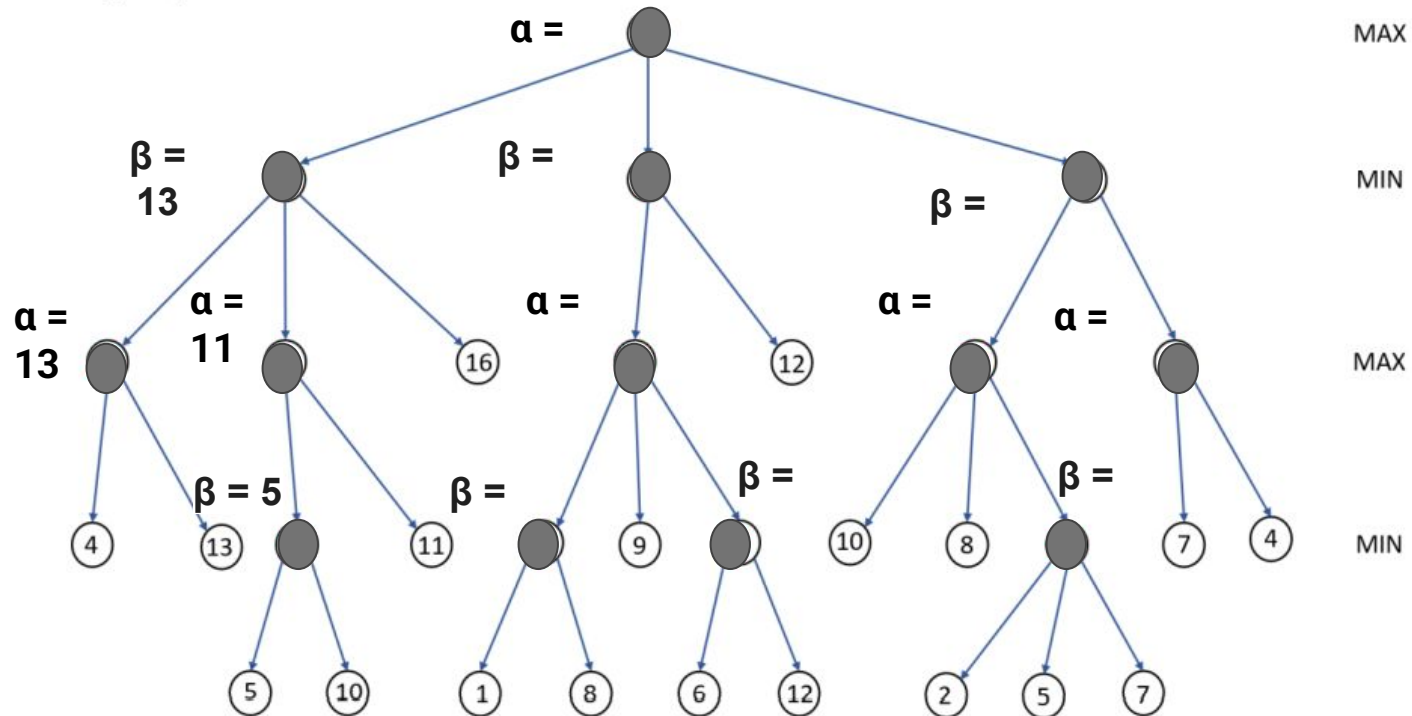
Use the Alpha-Beta pruning algorithm to prune the game tree below. Assuming child nodes are visited from left to right.



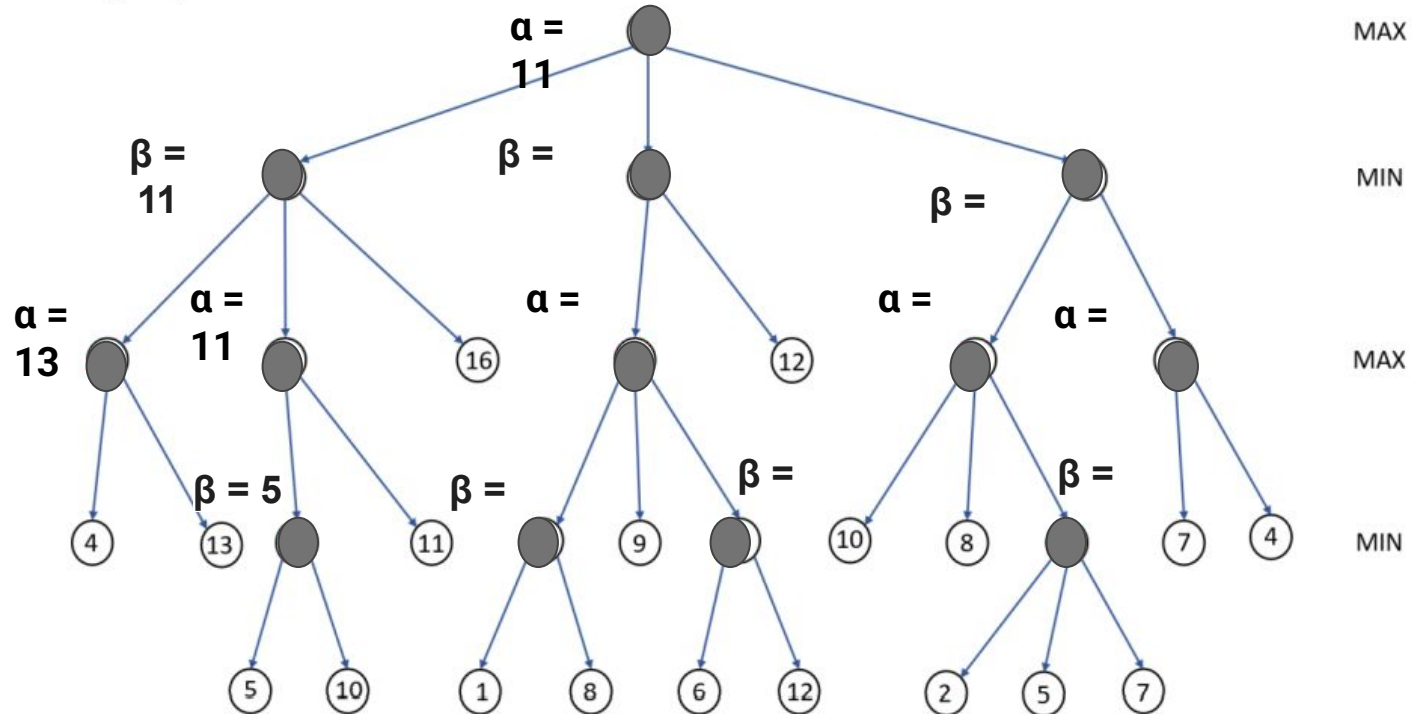
Use the Alpha-Beta pruning algorithm to prune the game tree below. Assuming child nodes are visited from left to right.



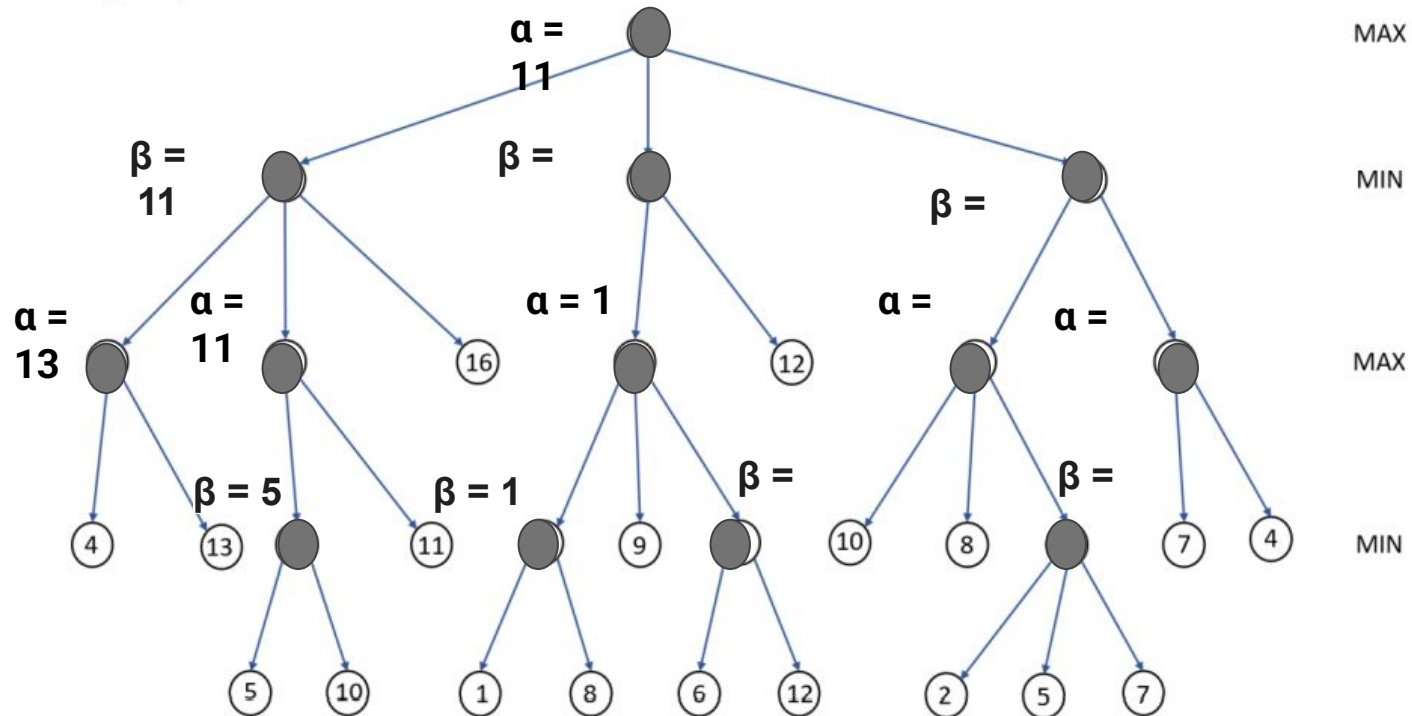
Use the Alpha-Beta pruning algorithm to prune the game tree below. Assuming child nodes are visited from left to right.



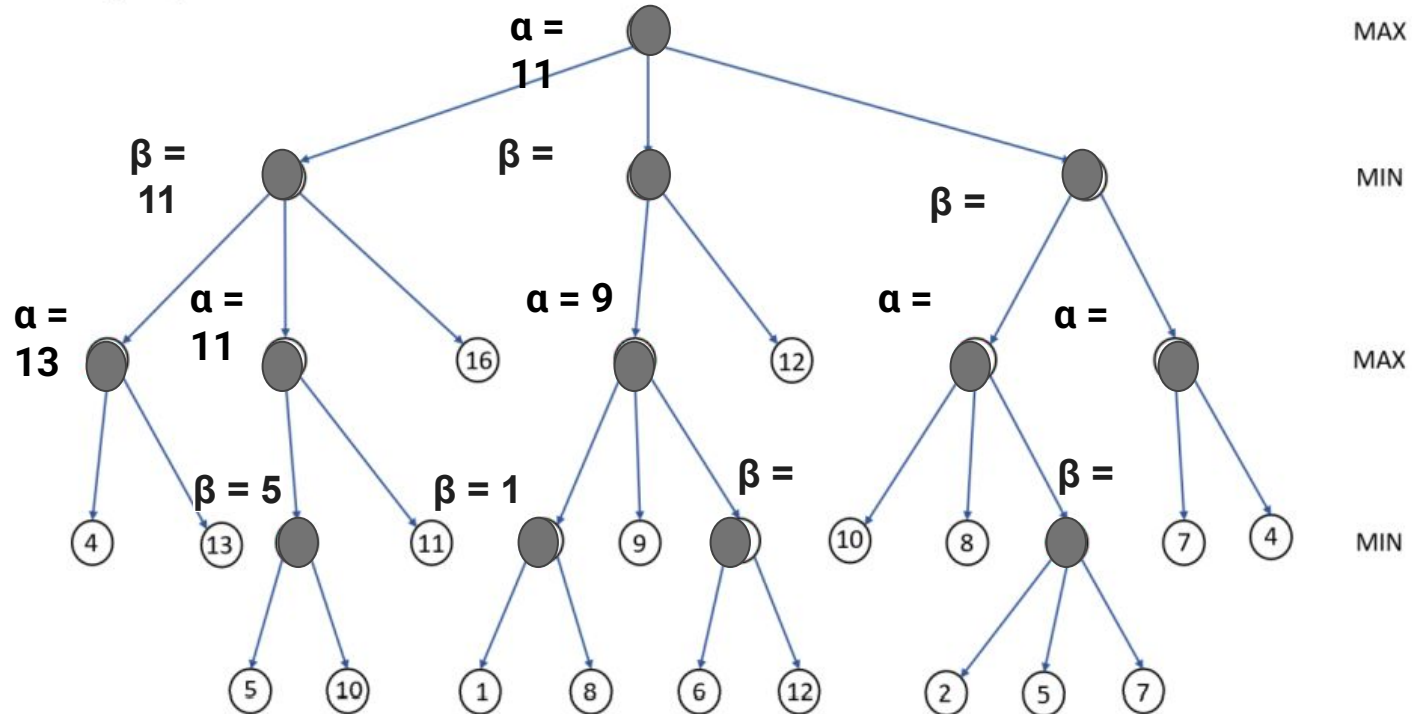
Use the Alpha-Beta pruning algorithm to prune the game tree below. Assuming child nodes are visited from left to right.



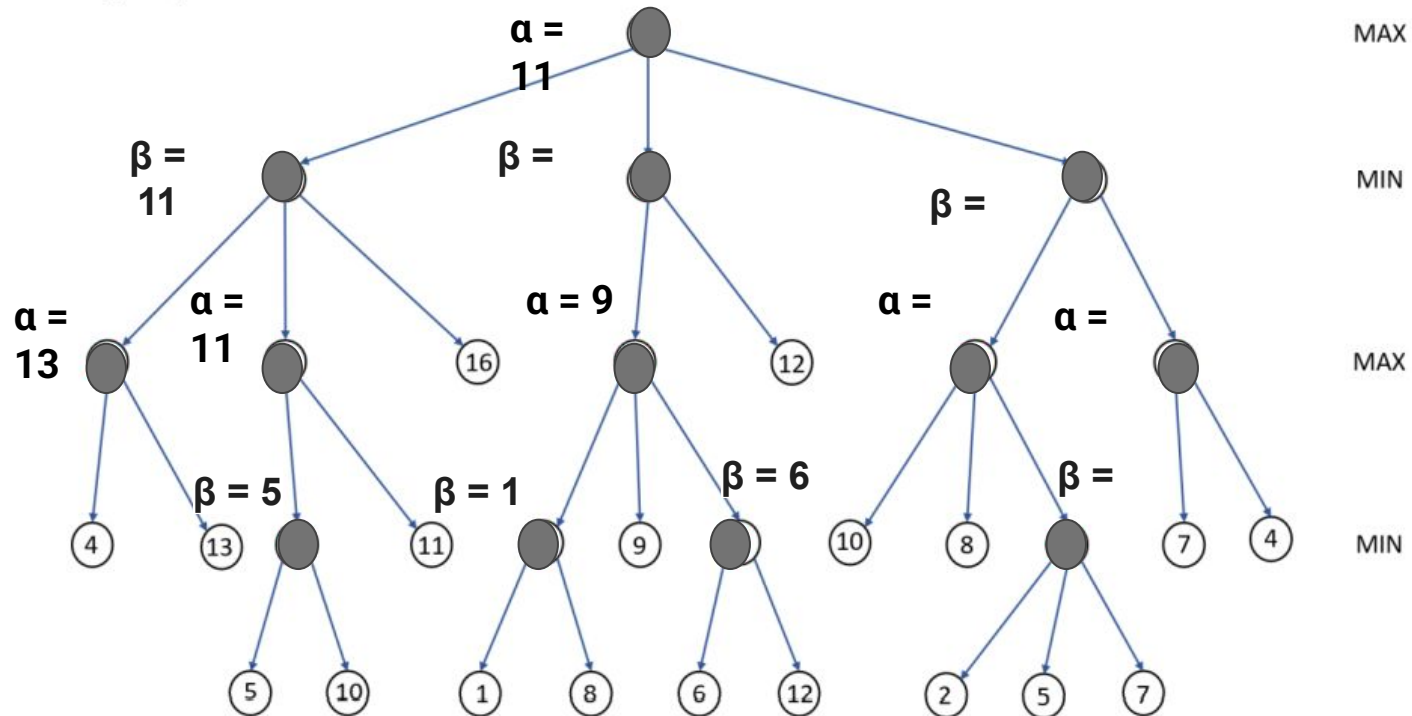
Use the Alpha-Beta pruning algorithm to prune the game tree below. Assuming child nodes are visited from left to right.



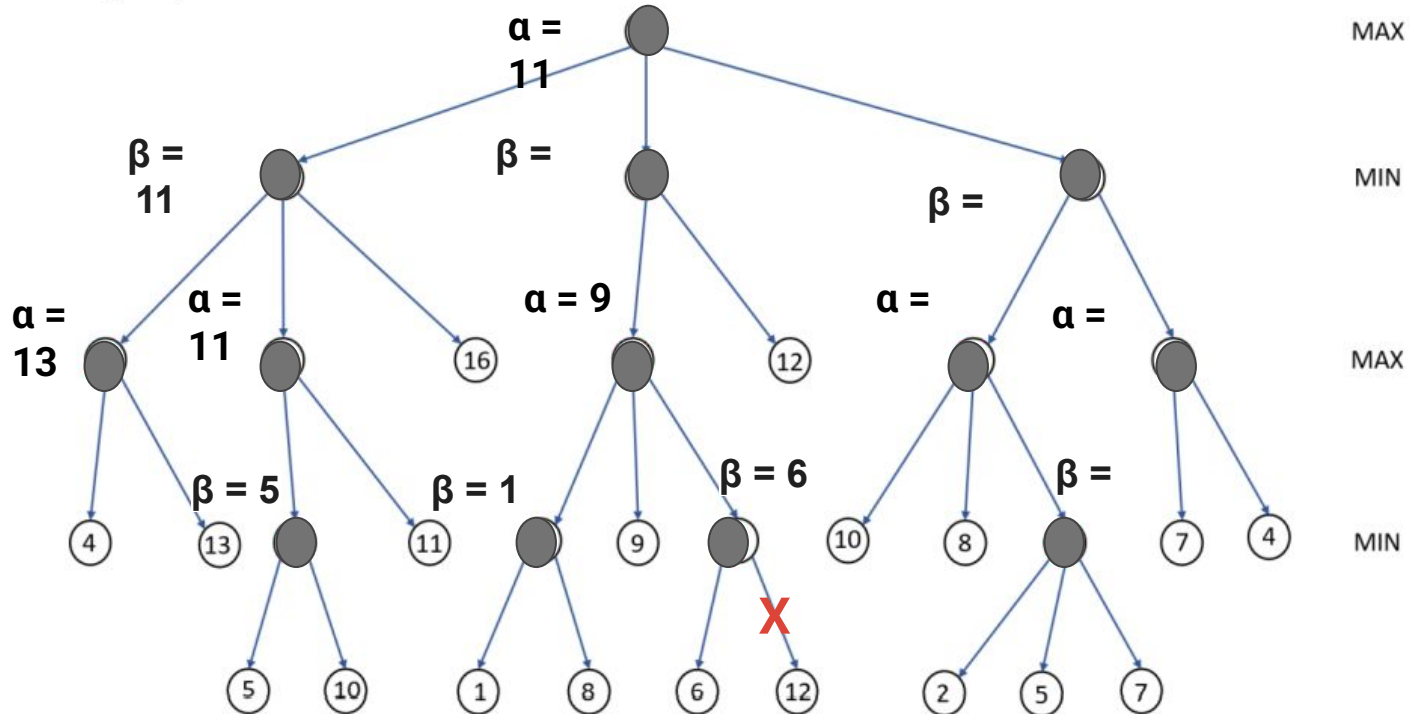
Use the Alpha-Beta pruning algorithm to prune the game tree below. Assuming child nodes are visited from left to right.



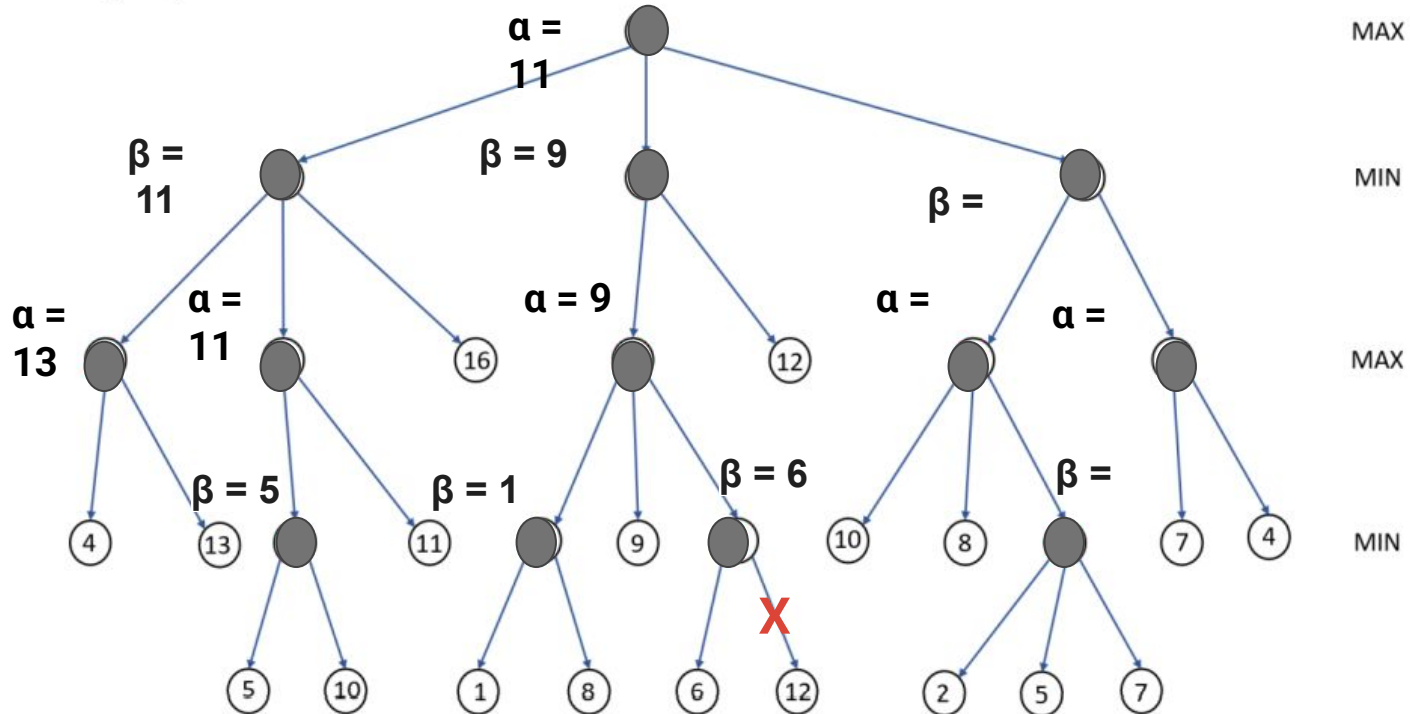
Use the Alpha-Beta pruning algorithm to prune the game tree below. Assuming child nodes are visited from left to right.



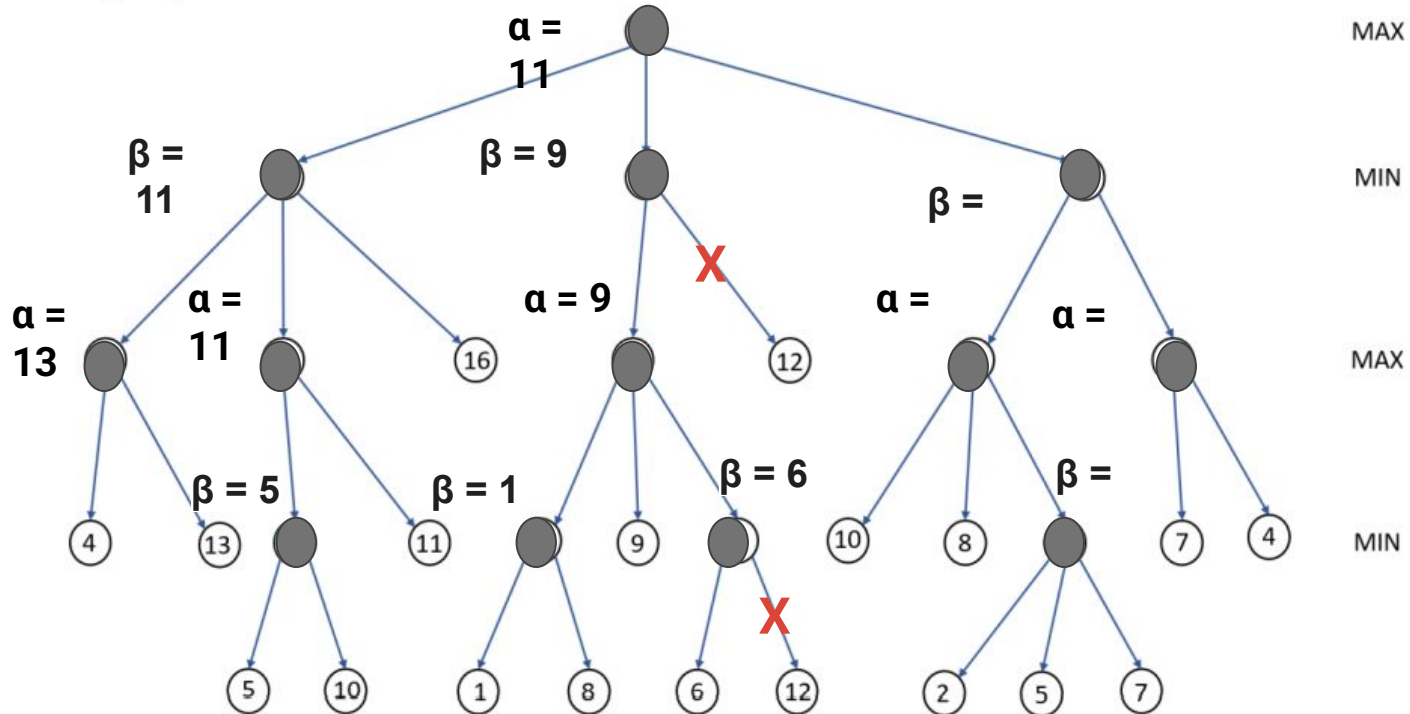
Use the Alpha-Beta pruning algorithm to prune the game tree below. Assuming child nodes are visited from left to right.



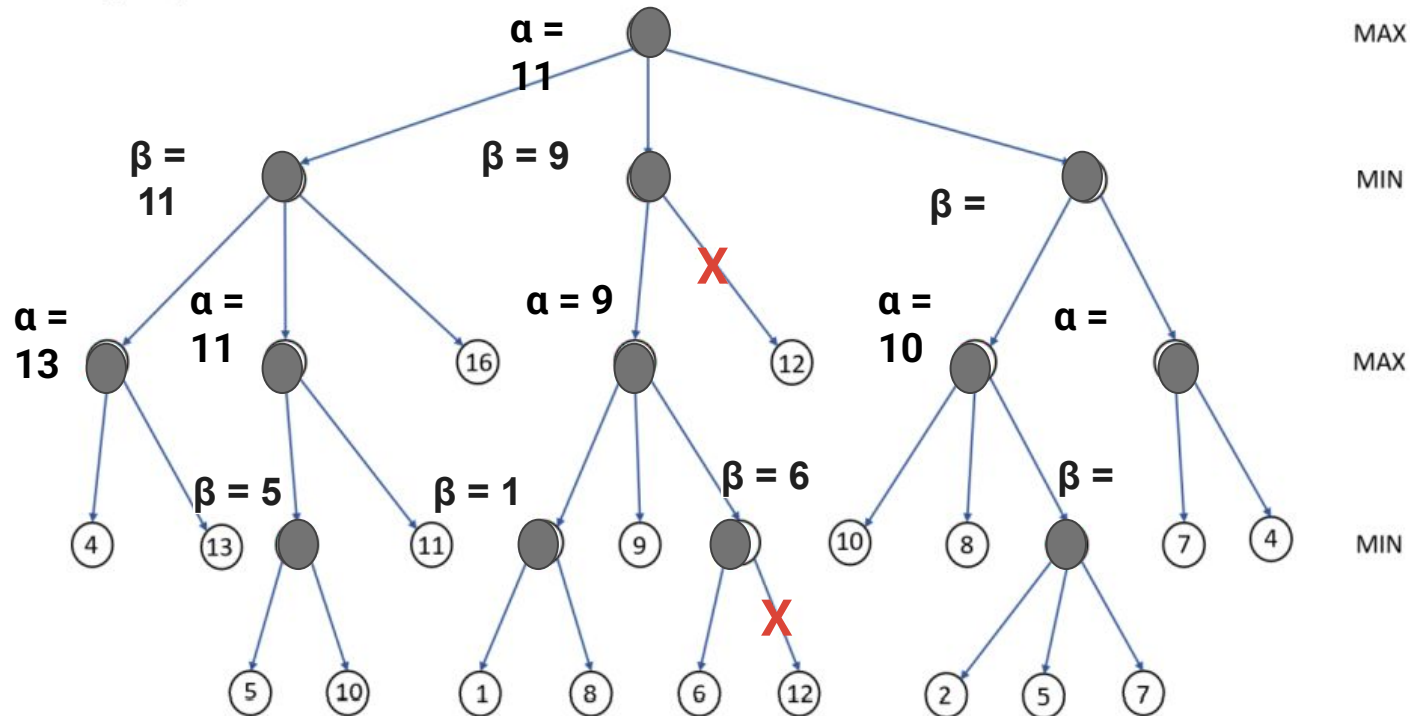
Use the Alpha-Beta pruning algorithm to prune the game tree below. Assuming child nodes are visited from left to right.



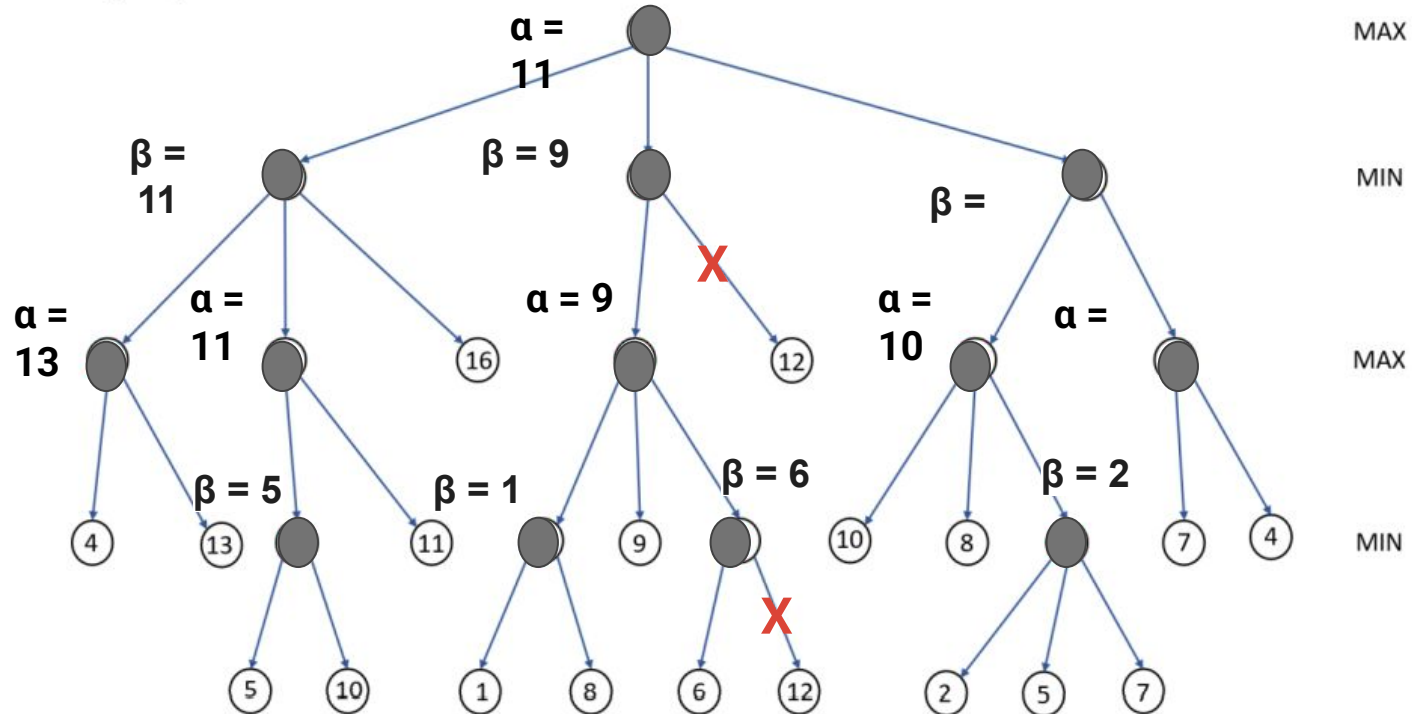
Use the Alpha-Beta pruning algorithm to prune the game tree below. Assuming child nodes are visited from left to right.



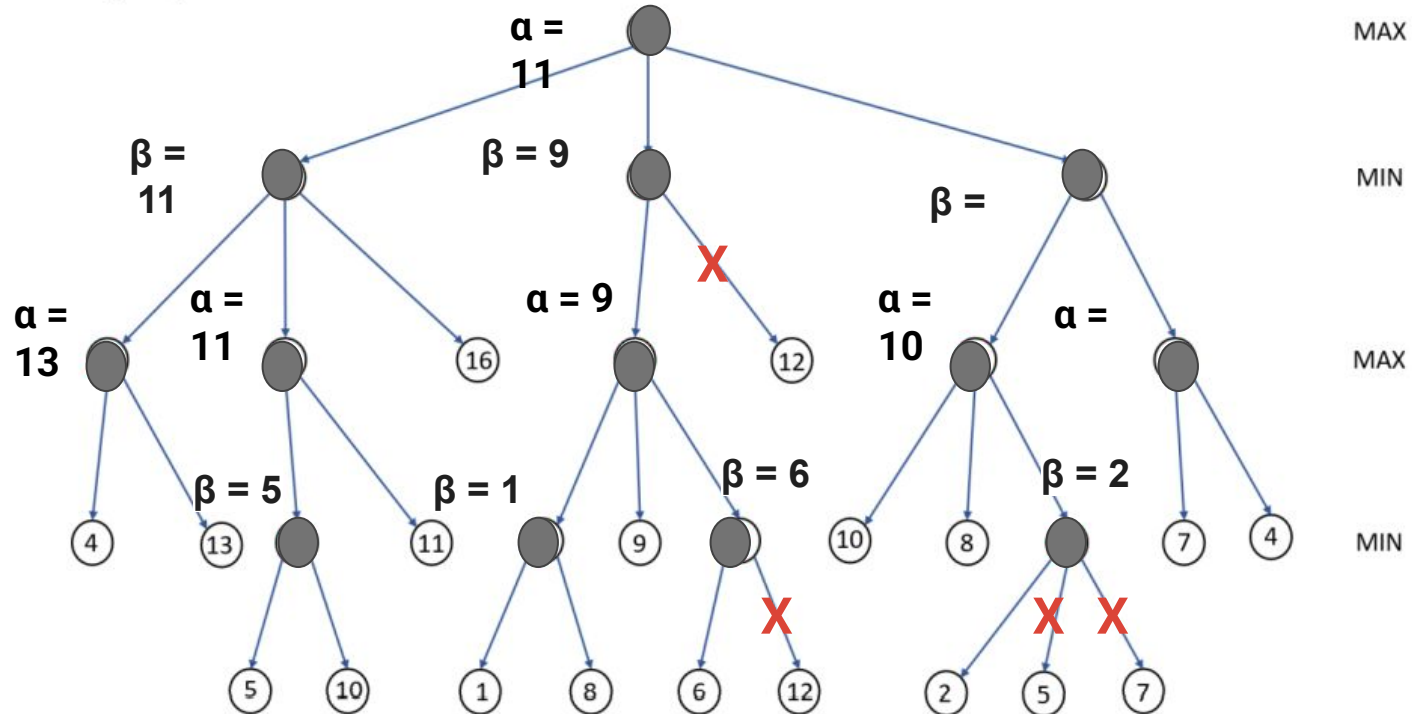
Use the Alpha-Beta pruning algorithm to prune the game tree below. Assuming child nodes are visited from left to right.



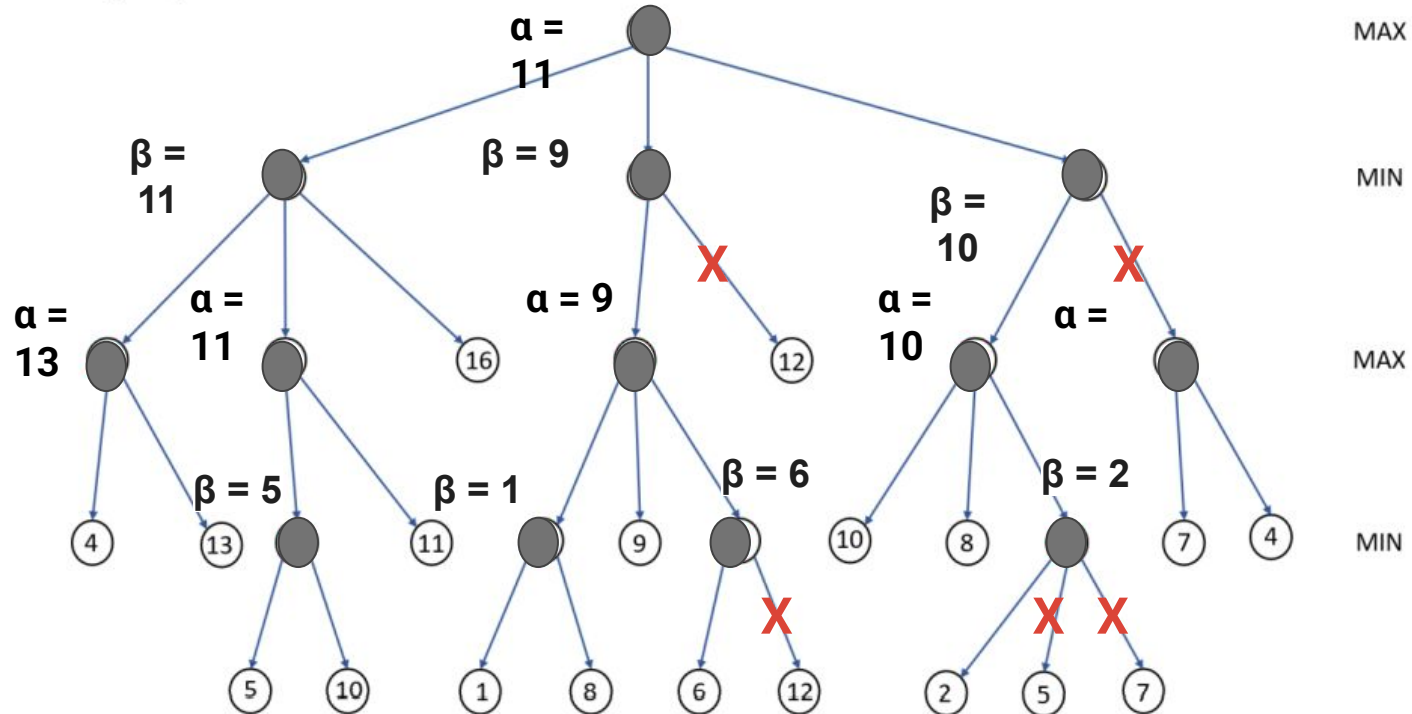
Use the Alpha-Beta pruning algorithm to prune the game tree below. Assuming child nodes are visited from left to right.



Use the Alpha-Beta pruning algorithm to prune the game tree below. Assuming child nodes are visited from left to right.



Use the Alpha-Beta pruning algorithm to prune the game tree below. Assuming child nodes are visited from left to right.



Heuristic Alpha–Beta Tree Search

- To make use of our limited computation time, we can cut off the search early and apply a heuristic evaluation function to states.
- Effectively treating non-terminal nodes as if they were terminal.
- In other words, we replace the UTILITY function with EVAL, which estimates a state's utility.
- We also replace the terminal test by a cutoff test, which must return true for terminal states, but is otherwise free to decide when to cut off the search

Heuristic Alpha–Beta Tree Search

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MIN} \end{cases}$$

$$\text{H-MINIMAX}(s, d) = \begin{cases} \text{EVAL}(s, \text{MAX}) & \text{if IS-CUTOFF}(s, d) \\ \max_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if TO-MOVE}(s) = \text{MIN}. \end{cases}$$

Minimax properties

Is minimax optimal? Is it complete?

Minimax properties

Is minimax optimal? Is it complete?

Time complexity = ?

Space complexity = ?

Minimax properties

Is minimax optimal? Is it complete?

Time complexity = $O(b^d)$

Space complexity = $O(bd)$

Minimax properties

Is minimax optimal? Is it complete?

Time complexity = $O(b^d)$

Space complexity = $O(bd)$

Is it practical? In chess, $b=35$, $d=100$

Minimax properties

Is minimax optimal? Is it complete?

Time complexity = $O(b^d)$

Space complexity = $O(bd)$

Is it practical? In chess **b=35, d=100**

$O(35^{100})$ is a big number...



Minimax properties

Is minimax optimal? Is it complete?

Time complexity = $O(b^d)$

Space complexity = $O(bd)$

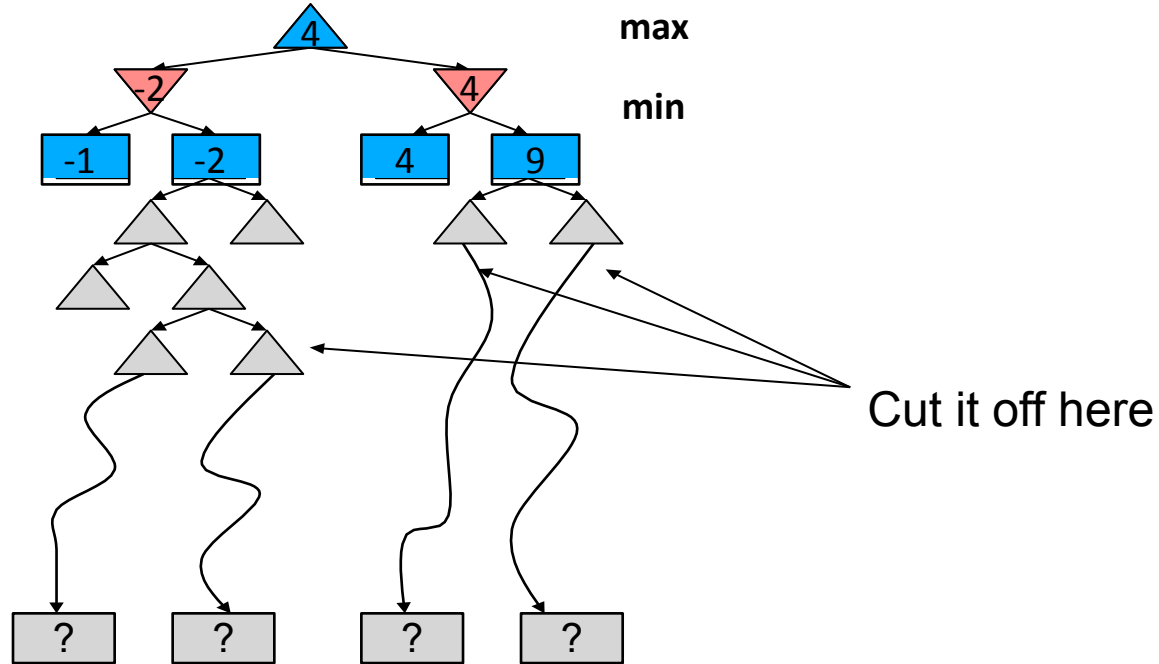
Is it practical? In chess **b=35, d=100**

$O(35^{100})$ is a big number...

So what can we do?

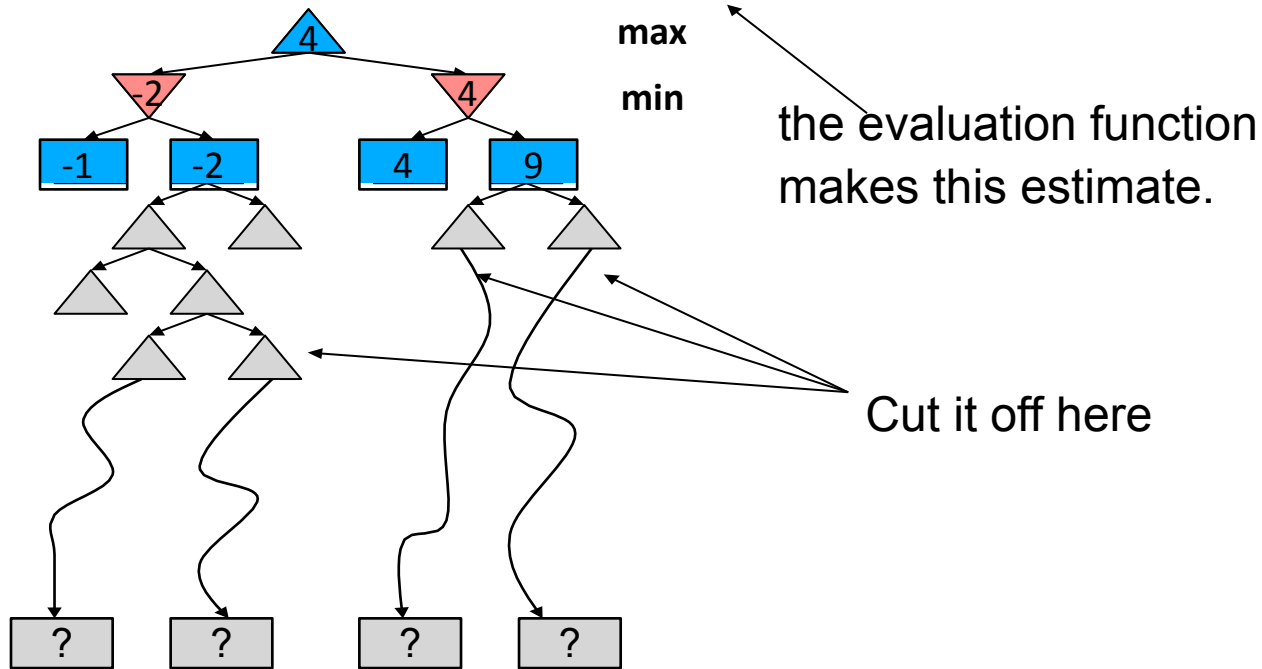
Evaluation functions

Key idea: cut off search at a certain depth and give the corresponding nodes an estimated value.



Evaluation functions

Key idea: cut off search at a certain depth and give the corresponding nodes an **estimated value**.



Evaluation functions

Problem: In realistic games, cannot search to leaves!

Solution: Depth-limited search

Instead, search only to a limited depth in the tree

Replace terminal utilities with an evaluation function

for non-terminal positions

Example:

Suppose we have 100 seconds

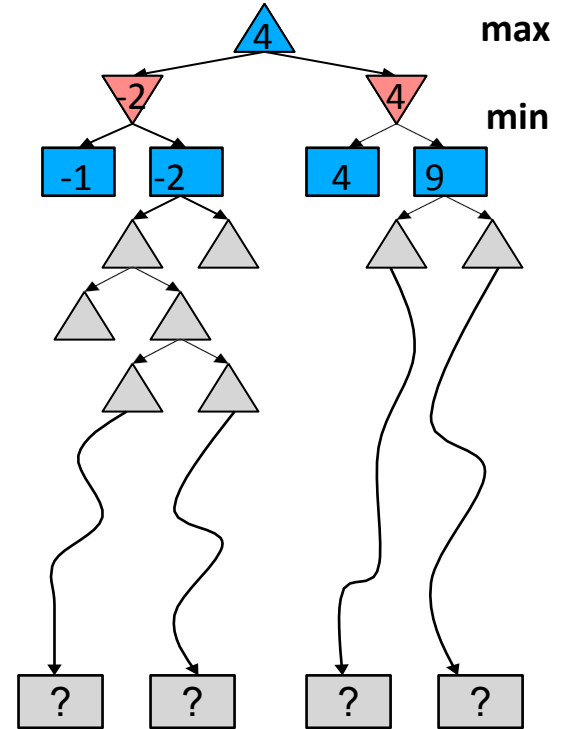
Can explore 10K nodes / sec

So can check 1M nodes per move

Guarantee of optimal play is gone

More plies makes a BIG difference

Use iterative deepening for an anytime algorithm



Evaluation functions

How does the evaluation function make the estimate? –
depends upon domain



For example, in chess, the value of a state
might equal the sum of piece values.

- a pawn counts for 1
- a rook counts for 5
- a knight counts for 3
- ...

A weighted linear evaluation function

$$eval(s) = w_1 f_1(s) + \dots + w_n f_n(s)$$

$f_1(s) \equiv$ number of pawns on the board

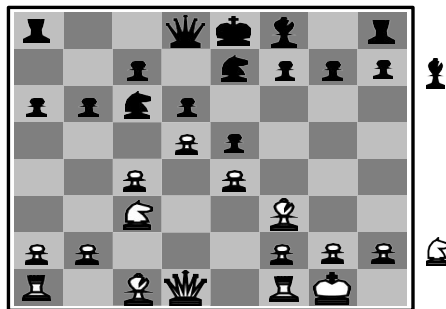
$f_2(s) \equiv$ number of knights on the board

$\vdots$

$w_1 = 1$ ← A pawn counts for 1

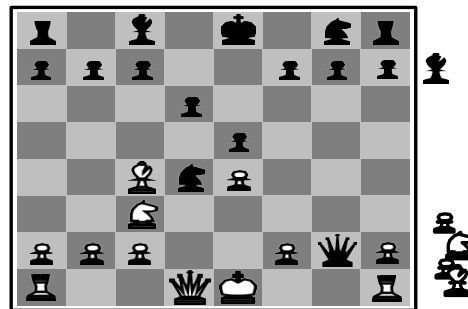
$w_2 = 3$ ← A knight counts for 3

$\vdots$



Black to move

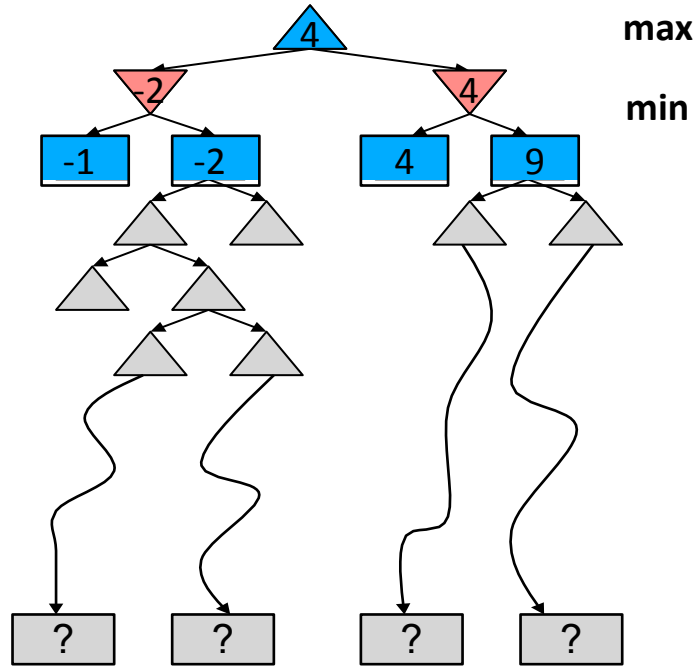
White slightly better



White to move

Black winning

At what depth do you run the evaluation function?



Option 1: cut off search at a fixed depth

Option 2: cut off search at particular states deeper than a certain threshold

The deeper your threshold, the less the quality of the evaluation function matters...

An Evaluation Function Based on Expected Outcomes

$$eval(s) = P(win) \times 1 + P(draw) \times \frac{1}{2} + P(loss) \times 0$$

If experience suggests the following probabilities for a in the two-pawns versus one-pawn category:

- Winning probability: $P(win)=0.82$
- Drawing probability: $P(draw)=0.16$
- Losing probability: $P(loss)=0.02$

Then the evaluation function computes:

An Evaluation Function Based on Expected Outcomes

$$eval(s) = P(win) \times 1 + P(draw) \times \frac{1}{2} + P(loss) \times 0$$

If experience suggests the following probabilities for a in the two-pawns versus one-pawn category:

- Winning probability: $P(win)=0.82$
- Drawing probability: $P(draw)=0.16$
- Losing probability: $P(loss)=0.02$

Then the evaluation function computes:

$$eval(s) = (0.82 \times 1) + (0.16 \times \frac{1}{2}) + (0.02 \times 0) = 0.90$$